

BRAHMITOKENIZER-131K: AN INDIC-CAPABLE DROP-IN REPLACEMENT FOR o200k_BASE

TECHNICAL REPORT

Rohan Shravan

The School of AI
India

rshravan@theschoolofai.in

May 29, 2026

ABSTRACT

We present BrahmicTokenizer-131K, a 131,072-vocabulary byte-level BPE tokenizer that closes the Brahmic compression gap at the 131K-vocabulary class while preserving the English, EU-language, and code compression of OpenAI’s o200k_base. We construct it through a two-stage retrofit of o200k_base: (1) a script-prune crop that reduces 200,019 tokens to 131,072 by removing tokens covering nine out-of-scope writing systems, and (2) a surgical retrofit of 2,372 corpus-dead vocabulary slots determined by linear-programming allocation across nine Brahmic Unicode blocks. The pre-tokenizer, decoder, and inherited merge rules are unchanged from o200k_base, making BrahmicTokenizer-131K a drop-in replacement at the tokenizer interface: a training pipeline using o200k_base can swap in our `tokenizer.json` without modifying data loader, pre-tokenizer, or decoder code. The embedding matrix resizes from 200,019 to 131,072 rows — a standard vocabulary-resize operation that downstream training already handles.

On 27 million documents of public Indic pretraining text (2.84 billion words, 46.21 GB), BrahmicTokenizer-131K produces 26.7% fewer tokens than Mistral-Nemo Tekken / Sarvam-m at the same vocabulary budget, with per-language savings of 15.79% (Tamil) to 76.79% (Odia, a $4.31\times$ compression ratio). The Odia advantage is mechanistically explained by Tekken/Sarvam-m containing zero Oriya-block tokens; our surgery added 725. On non-Indic content, BrahmicTokenizer-131K matches o200k_base’s English fertility (1.235 versus 1.232 tokens/word) and beats Tekken/Sarvam-m by 4.0–14.2% on HumanEval, MBPP, and GSM8K. Across our 14-tokenizer benchmark, BrahmicTokenizer-131K is the only tokenizer simultaneously competitive on Brahmic, English, EU, code, and math evaluation at the 131K vocabulary budget. Specialist tokenizers at other vocab classes (Sarvam-30B at 262K, Sarvam-1 at 68K, MUTANT-Indic) achieve better Indic compression at the cost of non-Indic performance: Sarvam-1’s English fertility is 15.9% worse and its code/math compression is 26–33% worse than ours. We release the artifact under Apache 2.0 license at <https://huggingface.co/theschoolofai/BrahmicTokenizer-131K>.

Keywords tokenizer, byte-level BPE, Brahmic scripts, Indic languages, multilingual NLP, o200k_base

1 Introduction

Tokenizer choice determines, on real Brahmic web text, whether a 37-character Odia sentence becomes 21 tokens or 99 tokens. The first number is what BrahmicTokenizer-131K produces on the sentence ଓଡ଼ିଆ ଭାଷା ଓଡ଼ିଶା ରାଜ୍ୟର ସରକାରୀ ଭାଷା (“Odia is the official language of Odisha state”). The second is what Tekken/Sarvam-m produces on the same sentence at the same vocabulary budget (131K tokens for both). The $4.7\times$ gap is not a curated single-example artifact: on the 60.3-million-word Odia portion of a 27-million-document Indic pretraining corpus, the ratio holds at $4.31\times$. The structural cause is verifiable: Tekken/Sarvam-m contains zero tokens from the Oriya Unicode block (U+0B00–U+0B7F) in its 131K vocabulary, so every Odia character falls to byte-level fallback at three tokens per character.

This same pattern, with different magnitudes, repeats across all eleven Brahmic-script Indian languages. Tokens-per-word fertility on FLORES-200 (Costa-jussà et al., 2022) reaches 16.78 for Llama-3.1-8B (Grattafiori et al., 2024; Meta AI, 2024) on Odia (versus 1.24 for English), 18.18 for Tekken/Sarvam-m (Mistral AI, 2024; Sarvam AI, 2025b), and 6.79 for GPT-OSS-120B (the inheritor of o200k_base) (OpenAI, 2025) despite o200k_base (OpenAI, 2024) being the strongest open tokenizer for English and code at the 200K-vocab tier. The gap is large, persistent across tokenizer generations, structurally verifiable from per-tokenizer vocabulary contents, and corresponds to a measurable training-compute cost on real Indic pretraining corpora. Cross-language tokenization disparities of this kind are a well-documented source of cost and capability inequality in modern LLMs (Petrov et al., 2023; Ahia et al., 2023); this paper closes the gap at the 131K-vocabulary budget while preserving the English, EU-language, and code compression of the o200k_base family bit-identically.

Open-vocabulary tokenizers determine the units that every downstream component of a large language model operates on: the granularity of the gradient signal at each embedding row, the effective context length of a fixed-token-budget window, the cost of training and inference per character, and the floor on a model’s multilingual capability. For the nine Brahmic scripts of South Asia, used by approximately 1.4 billion people across 11 major Indian languages, the dominant tokenizers in production are inadequate, and the inadequacy is not a translation-quality artifact but a structural property of how their vocabularies were trained.

This paper introduces **BrahmicTokenizer-131K**, a 131,072-vocabulary byte-level BPE tokenizer that closes the Brahmic compression gap while preserving the English, EU-language, and code compression of the o200k_base family. The design target is general-purpose multilingual coverage for English, three major European languages (French, German, Spanish), program source code, mathematical notation, and the nine Brahmic scripts that encode 11 Indian languages (Hindi, Marathi → Devanagari; Bengali, Assamese → Bengali script; Tamil; Telugu; Kannada; Malayalam; Gujarati; Punjabi → Gurmukhi; Odia → Oriya). We construct the tokenizer in two stages from OpenAI’s o200k_base: a script-prune crop that reduces the 200,019-token base vocabulary to 131,072 by removing the 38,345 tokens covering nine non-target scripts, and a surgical retrofit of 2,372 corpus-dead slots in the resulting cropped vocabulary with high-frequency Brahmic content. The pre-tokenizer, decoder, and English/EU/code merge rules are inherited unchanged from o200k_base.

We evaluate BrahmicTokenizer-131K against 13 publicly available tokenizers spanning four vocabulary classes (48K to 262K) and four pre-tokenizer types on five evaluation axes: token volume on a 27-million-document Indic pretraining corpus, per-word fertility on FLORES-200 and IN22-Gen (Gala et al., 2023) across 22 languages, bytes-per-token compression, structural diagnostics, and code/math compression. The evidence supports six empirically-grounded claims:

1. Substantial Indic compression advantage at the 131K vocab budget. On 27 million documents (2.84 billion words, 46.21 GB) of public Indic pretraining text, BrahmicTokenizer-131K produces 26.7% fewer tokens than Tekken/Sarvam-m (the same 131K-vocab multilingual tokenizer pair). The per-language advantage holds on 11 of 11 Brahmic languages, ranging from 15.79% (Tamil) to 76.79% (Odia, a $4.31\times$ compression ratio). For a training pipeline targeting Indic-heavy corpora, this is a 27% per-step compute reduction.

2. English compression at o200k-grade. BrahmicTokenizer-131K achieves 1.235 tokens/word on FLORES-200 English (versus o200k_base’s 1.232 and Tekken/Sarvam-m’s 1.267, a 2.5% advantage over Tekken/Sarvam-m). On a 1,000-document English corpus the agreement with o200k_cropped is byte-identical on 96.8% of documents and within 0.034% in total token count. The surgery preserved English compression to the precision a downstream training pipeline can observe.

3. Code and math compression best-in-class. BrahmicTokenizer-131K achieves 0.295 tokens/character on HumanEval (Chen et al., 2021), 0.320 on MBPP (Austin et al., 2021), and 0.301 on GSM8K (Cobbe et al., 2021). Against Tekken/Sarvam-m at the same vocabulary budget, BrahmicTokenizer-131K wins by 4.0%, 5.4%, and 14.2% respectively. Against Sarvam-30B (Sarvam AI, 2025a) at $2\times$ our vocabulary budget, BrahmicTokenizer-131K wins by 13.2%, 21.3%, and 16.6%. The GSM8K advantage in particular reflects o200k_base’s individual-digit tokenization, which the surgery preserved unchanged.

4. EU language compression competitive. BrahmicTokenizer-131K achieves French 1.464, German 1.653, Spanish 1.388 tokens/word on FLORES-200. Against Tekken/Sarvam-m: tied on German, +2.2% on French, +1.2% on Spanish, within 2.5% of best on each. Against the larger Sarvam-30B vocabulary: within 3.0%. EU language compression was not a primary design target, but the surgery’s preservation of o200k_base’s Latin BPE merges yielded competitive performance.

5. Structural properties preserved. The two-stage construction preserves structural properties of o200k_base relevant for byte-pooled embedding architectures: (a) maximum token byte length of 32 UTF-8 bytes, (b) zero tokens spanning two disjoint writing systems, and (c) the GPT-2 ByteLevel pre-tokenizer (Radford et al., 2019) and decoder

unchanged. The byte-length and script-purity properties are inherited from the Stage-1 script-prune crop (which incidentally removed every o200k_base token violating these properties) and preserved by Stage-2’s no-cross-script-merge rule. Among 14 publicly-available tokenizers we benchmark, BrahmicTokenizer-131K and o200k_cropped are the only two with both properties satisfied (Appendix A).

6. General-purpose at the 131K vocabulary budget. Among publicly available tokenizers at the 131K-vocab class, BrahmicTokenizer-131K is the only tokenizer simultaneously competitive on every evaluation axis: Indic compression matching or exceeding all general-purpose 131K-class baselines, English compression matching o200k_base, code and math compression exceeding every comparator we tested, and EU language compression within 3% of best. Specialist tokenizers at other vocabulary classes outperform BrahmicTokenizer-131K on their specialty (Sarvam-1 (Sarvam AI, 2024a) at 68K wins on Indic by 12.7% on the same corpus; Sarvam-30B at 262K wins by 18.7% on Indic and on two of three EU languages) but lose on the breadth axis. Sarvam-1’s English fertility is 1.43 tokens/word (15.9% worse than ours) and its code/math compression is 26–33% worse; Sarvam-30B’s code/math compression is 13–21% worse than ours despite double the vocabulary. The 12.7% loss to Sarvam-1 on Indic and the 18.7% loss to Sarvam-30B on Indic are the costs of being general-purpose at the 131K budget; we consider those costs justified by the breadth of capability they buy.

Disambiguating “drop-in replacement.” The term has narrow meaning in this paper. BrahmicTokenizer-131K uses the same byte-level BPE algorithm, same GPT-2 ByteLevel pre-tokenizer, same decoder, same special-token format, and same vocabulary file format as o200k_base. A training pipeline currently using o200k_base can swap in BrahmicTokenizer-131K’s `tokenizer.json` and resume training without modifying any data-loader, pre-tokenizer, or decoder code. The embedding matrix and language-model head resize from 200,019 to 131,072 rows — a standard vocabulary-resize operation, mechanically identical to switching between any two BPE tokenizers of different sizes. The vocabulary content differs in $38,345 + 2,372 = 40,717$ of 131,072 slots, but the tokenizer-side interface (algorithm, pre-tokenizer regex, decoder, special-token format, JSON schema) is identical. The English, EU, and code merge rules are inherited unchanged from o200k_base; the Brahmic merge rules are new.

Methodological contribution. We document the surgical retrofit construction as a methodology for vocabulary expansion in any byte-level BPE tokenizer where (a) the source tokenizer is well-engineered for a target subset of languages, and (b) the target deployment needs additional language coverage beyond what frequency-based BPE training on a corpus would produce. The construction is substantially more compute-efficient than training a tokenizer from scratch on a Brahmic-heavy corpus, while preserving all source-tokenizer properties by construction and avoiding the risk of frequency-based BPE underproducing rare-but-important content classes. We also document Stage-1’s incidental enforcement of byte-length and cross-script-purity properties, and the per-script LP allocation policy that determined the 2,372-slot vocabulary surgery.

Artifact contribution. We release BrahmicTokenizer-131K under Apache 2.0 license at <https://huggingface.co/theschoolofai/BrahmicTokenizer-131K> and <https://github.com/theschoolofai/BrahmicTokenizer-131K>, including the `tokenizer.json` file, the 23-test internal audit suite, the four reviewer-runnable verification scripts (Section 4.5), the per-language LP allocation tables (Section 3.5.1), and the structural-comparison tables for the 14-tokenizer benchmark (Appendix A). The release includes reproduction scripts for FLORES-200, IN22-Gen, HumanEval, MBPP, and GSM8K evaluation across all 14 tokenizers. Large public datasets that are already downloadable elsewhere (the AI4Bharat Indic stack, FLORES-200, IN22-Gen, HumanEval, MBPP, GSM8K) are not redistributed; the reproduction scripts download them from their canonical sources. The shipped tokenizer is a derivative of o200k_base (OpenAI, 2024), released through the MIT-licensed `tiktoken` repository; Apache 2.0 is compatible with incorporating MIT-licensed material. The bundled Brahmic-script fonts (NotoSansDevanagari, NotoSansBengali, NotoSansOriya, NotoSansTamil) used to typeset this paper are redistributed under the SIL Open Font License 1.1.

Roadmap. Section 2 reviews relevant prior tokenizer work and frames the three-regime model of Brahmic compression. Section 3 describes the two-stage construction in detail. Section 4 reports internal validation. Section 5 reports external evaluation across the five axes. Section 6 reports ablations of the construction choices. Section 7 discusses limitations and future work. Section 8 concludes.

2 Background

2.1 Byte-level BPE

Byte-pair encoding (Sennrich et al., 2016; Gage, 1994) learns a vocabulary by iteratively merging the most-frequent adjacent byte pairs in a training corpus. Byte-level BPE, introduced in GPT-2 (Radford et al., 2019), operates on UTF-8 byte sequences rather than Unicode characters, which makes it script-agnostic at the input layer. Modern tokenizers

in production (o200k_base, Llama-3, Mistral Tekken, Sarvam-m) are byte-level BPE variants differing primarily in their pre-tokenizer, vocabulary size, and training corpus composition.

2.2 The GPT-2 ByteLevel pre-tokenizer

The GPT-2 ByteLevel pre-tokenizer applies a bijection from UTF-8 bytes to printable Unicode code points before BPE training. This bijection maps each of the 256 possible byte values to a printable character (avoiding whitespace and control characters that interfere with regex-based tokenization). The bijection has a critical implication for Brahmic scripts: the three-byte UTF-8 encoding of a Devanagari character (e.g., ऋ = 0xE0 0xA4 0xA8) becomes a three-character sequence (à, ¤, ") before BPE training. If the BPE training process does not see enough Devanagari content to form merges combining these three characters into single tokens, every Devanagari character at inference time will produce three tokens. This is the byte-fallback regime.

2.3 The three-regime model of Brahmic compression

We classify Brahmic compression by tokenizer into three regimes:

Byte-fallback regime. The tokenizer contains no merge rules combining the three UTF-8 bytes of any Brahmic character into a single token. Result: 3+ tokens per Brahmic character. Tekken/Sarvam-m falls into this regime on Odia; Llama-3.1-8B and Qwen3-8B fall into this regime on most Brahmic scripts.

Subword regime. The tokenizer contains merges that combine character-level Brahmic bytes into character tokens, plus subword pieces below word level. Result: roughly 2–3 tokens per Brahmic word, but variable by language. Most well-trained multilingual tokenizers operate in this regime on most Brahmic scripts.

Whole-word regime. The tokenizer contains single-token entries for high-frequency Brahmic words. Result: 1–2 tokens per common word. Indic-specialist tokenizers (Sarvam-1, MUTANT-Indic) achieve this regime for their target languages.

BrahmicTokenizer-131K aims for the subword regime for most Brahmic content plus the whole-word regime for the most frequent Brahmic words. The 2,372-slot surgery allocates these whole-word entries via the LP policy described in Section 3.5.1.

2.4 Prior tokenizers

We compare BrahmicTokenizer-131K against 13 publicly available tokenizers spanning four vocabulary classes:

General-purpose multilingual tokenizers: o200k_base (OpenAI, 2024) (200K vocab, GPT-2 ByteLevel pre-tokenizer, the reference for our retrofit), GPT-OSS-120B (OpenAI, 2025) (200K, structurally identical to o200k_base), Llama-3.1-8B (Grattafiori et al., 2024; Meta AI, 2024) (128K, ByteLevel), Qwen3-8B (Yang et al., 2025; Qwen Team, 2025) (152K, ByteLevel), DeepSeek-R1 (DeepSeek-AI, 2025; DeepSeek AI, 2025) (129K, ByteLevel), Gemma-3-1B (Gemma Team, 2025; Google DeepMind, 2025) (262K, Split pre-tokenizer), Mistral-Nemo Tekken (Mistral AI, 2024) (131K, ByteLevel).

Indic-capable variants: Sarvam-m (Sarvam AI, 2025b) (131K, ByteLevel, structurally identical to Tekken at the merge-table level), Sarvam-1 (Sarvam AI, 2024a) (68K, Metaspace pre-tokenizer, Indic-only), Sarvam-30B (Sarvam AI, 2025a) (262K, Split, Brahmic-heavy allocation), Krutrim-1 (Krutrim AI Labs, 2024) (70K, ByteLevel, Indic-focused), IndicBERTv2 (Doddapaneni et al., 2023) (200K, token-classification tokenizer), Airavata (Gala et al., 2024) (48K, Hindi-instruction-tuned, no pre-tokenizer).

Our 14-tokenizer benchmark includes BrahmicTokenizer-131K plus the 13 tokenizers above. We could not benchmark MUTANT-Indic directly because the tokenizer artifact was not publicly available at our paper preparation date (see Section 2.5).

2.5 MUTANT-Indic: the closest published methodological precedent

MUTANT-Indic (Rana et al., 2026) is the Indic-specific instance of the MUTANT multilingual tokenizer training recipe from Krutrim AI. Evaluated across English, 22 Indian languages, and code, the published paper claims a 39.5% average fertility improvement over Llama-4 and an 18% improvement over SUTRA (Bendale et al., 2024). The method combines language-aware pre-tokenization with a two-stage subword-then-multiword learning process inspired by SuperBPE (Liu et al., 2025). The MUTANT recipe is the closest methodological precedent for our work in spirit: both papers operate on the premise that the standard pre-tokenizer and BPE pipeline can be improved for Indic-script efficiency. Where we differ: MUTANT-Indic trains from scratch on Indic-heavy data; we retrofit from an English-strong

baseline. The two approaches optimize for different deployment profiles (Indic-specialist vs. drop-in for o200k_base). At the time of our paper preparation, the MUTANT-Indic tokenizer artifact was not publicly available for download, so we cite the published numbers without independent benchmarking on our shared evaluation corpora.

2.6 Vocabulary expansion and tokenizer transfer

Adding script coverage to an existing tokenizer rather than training one from scratch has prior art in the cross-lingual transfer literature. Cui et al. (2023) expand LLaMA’s vocabulary with new Chinese tokens and demonstrate efficient continued pretraining on the augmented vocabulary; this is the closest existing precedent for the vocabulary-surgery approach we use. Minixhofer et al. (2022) address the related problem of replacing a model’s tokenizer wholesale and re-initializing embeddings for cross-lingual transfer. Our work differs from both: we do not replace the tokenizer (the merge rules and pre-tokenizer are preserved bit-identically on non-Indic content), and we do not perform model-side training to consume the new vocabulary — the artifact ships as a drop-in for o200k_base such that a downstream training pipeline absorbs the new entries through its normal embedding-learning dynamics. The construction is therefore a pure tokenizer-side intervention that is orthogonal to, and composable with, the model-side transfer techniques of Cui et al. (2023) and Minixhofer et al. (2022).

3 Method

3.1 Two-stage overview

BrahmicTokenizer-131K is constructed in two stages from o200k_base. Stage 1 produces an intermediate tokenizer we call o200k_cropped via a script-prune crop. Stage 2 retrofits 2,372 corpus-dead slots in o200k_cropped with high-frequency Brahmic content. The construction is reproducible from o200k_base’s published vocabulary plus our audit corpus.

3.2 Stage 1: script-prune crop

Stage 1 reduces o200k_base’s 200,019-token vocabulary to 131,072 by removing 38,345 tokens covering nine non-target scripts: CJK Unified Ideographs, Hangul (Korean), Hiragana+Katakana (Japanese), Arabic, Cyrillic, Thai, Greek, Hebrew, and Sinhala. The remaining 130,716 normal tokens plus 356 special tokens form o200k_cropped. Table 1 shows the breakdown by removed script.

Table 1: Stage-1 script-prune: 38,345 tokens removed across 9 scripts.

Removed script	Token count
CJK Unified Ideographs	15,872
Hangul (Korean)	6,401
Hiragana + Katakana (Japanese)	4,937
Arabic	3,204
Cyrillic	2,856
Thai	1,561
Greek	1,722
Hebrew	1,503
Sinhala	289
Total	38,345

The script-prune crop incidentally removes every o200k_base token longer than 32 UTF-8 bytes (266 tokens) and every token spanning two disjoint writing systems (59 tokens). The Stage-2 surgery preserves both properties by construction via the no-cross-script-merge rule (Section 3.6). These properties are relevant for byte-pooled embedding architectures that operate on token byte sequences; we report them as structural observations without claiming a specific architectural application.

3.3 Audit corpus

We assembled a 1.045-billion-token audit corpus from the public AI4Bharat Indic stack (Sangraha (Khan et al., 2024) monolingual subsets, Bharat Parallel Corpus Collection, Samanantar (Ramesh et al., 2022), NLLB-filtered, IndicCom-

parable, ILCI (Jha, 2010), AI4Bharat Wikipedia dumps) plus Sarvam-AI’s Samvaad-Hi conversational corpus (Sarvam AI, 2024b). The audit corpus was used to (a) measure token fire rates in o200k_cropped to identify dead slots, and (b) score per-script saturation curves for the LP allocation. Per-language coverage was weighted by approximate per-language frequency in the underlying training data.

3.4 Drop decisions

We tokenized the 1.045-billion-token audit corpus with o200k_cropped and identified 2,372 tokens with audit fire rate of zero, plus a small number of marginal cases (fire rate $\leq 1,000$ per billion tokens) that we removed by policy because they covered out-of-scope content (Sinhala, certain CJK transliterations). The signal-to-noise gap between median kept and median dropped tokens is approximately $197,000\times$. See Section 6.2 for the histogram.

3.5 Surgery composition

The 2,372 surgery slots are allocated to three categories:

Character infrastructure (5 slots): Five space-prefixed byte-pair intermediates that repair the GPT-2 ByteLevel pre-tokenizer’s three-byte UTF-8 encoding for high-frequency Brahmic scripts. Without these intermediates, common Brahmic characters require 3 byte-level fallback tokens. With them, common Brahmic characters become a single token after the pre-tokenizer’s ‘ $\tilde{G} + \grave{a}$ ’ merge has consumed the leading 0xE0 byte. Table 2 illustrates the effect on the worked example sentence.

Word-level slots (1,443 slots): High-frequency Brahmic words and subword pieces from the audit corpus. Approximately 95% of these slots are sub-word pieces (3 aksharas or fewer); 5% are full-word entries for the most frequent Brahmic words. The word-level allocation enables the whole-word regime described in Section 2.3.

Per-script Brahmic content (922 slots, plus 157 numerals/danda/artifacts): The remaining slots distributed across 9 Brahmic scripts by the LP allocation policy in Section 3.5.1. Plus 149 Indic-numeral merges and 3 shared Brahmic punctuation tokens.

Table 2: Three-way merge-trace comparison on the worked example sentence ଓଡ଼ିଆ ଭାଷା ଓଡ଼ିଶା ରାଜ୍ୟର ସରକାରୀ ଭାଷା (37 characters, “Odia is the official language of Odisha state”). Lower token count and lower broken-character count are better.

Tokenizer	Total tokens	Broken characters	% broken
BrahmicTokenizer-131K	21	0	0.0%
o200k_cropped (Stage-1 pre-surgery)	39	7	17.9%
Tekken/Sarvam-m	99	31	31.3%

3.5.1 Per-script LP allocation

The per-script allocation of 2,215 single-script Brahmic slots was determined by linear programming with two inputs: (a) per-script dead-slot supply from o200k_cropped, and (b) per-script under-representation relative to corpus token-share on the audit corpus. The LP objective maximized total token-savings on the audit corpus subject to per-script feasibility constraints.

Formulation. Let $S = \{1, \dots, 9\}$ index the nine Brahmic scripts and let $x_s \in \mathbb{Z}_{\geq 0}$ be the number of single-script slots allocated to script s . For each s , let $c_s: \mathbb{Z}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ be the saturation curve giving cumulative audit-corpus token savings as a function of slot count, computed by ranking the per-script candidate merges in descending order of audit fire count and summing fires up to x_s ; and let K_s be the per-script candidate-pool ceiling (the number of admissible single-script merges that survived the no-cross-script-merge filter of Section 3.6). The allocation solves

$$\max_{x \in \mathbb{Z}_{\geq 0}^9} \sum_{s \in S} c_s(x_s) \quad \text{subject to} \quad \sum_{s \in S} x_s = 2,215, \quad 0 \leq x_s \leq K_s \quad \forall s \in S. \quad (1)$$

Because each c_s is constructed as a cumulative sum over candidates pre-ranked by descending fire count, the curves are non-decreasing and concave by construction; later slots contribute by definition no more than earlier ones. The problem therefore reduces to a discrete concave-objective allocation and admits a greedy solver: repeatedly assign the next slot to the script with the largest current marginal $c_s(x_s + 1) - c_s(x_s)$, terminating when the total budget is exhausted. We invoke this solver on the per-script saturation curves and use its allocation as the LP-optimal reference. We validated the LP allocation against a hand-tuned heuristic and three alternative policies in Section 6.1; the LP and the heuristic

produced identical total savings at the 2,215-slot budget (the LP reaches a slightly different per-script allocation, since the optimum is flat over multiple allocations of equal total).

3.6 Merge rule generation

The Stage-2 surgery added 2,156 new merge-rule entries to o200k_cropped’s existing merge list and appended them after the inherited entries, producing BrahmicTokenizer-131K’s 301,398 total merge-rule entries (shipped `tokenizer.json`). New entries were generated by training BPE on the audit corpus restricted to the Brahmic-script subset and selecting the top-N merges that satisfied the no-cross-script-merge rule (no merge combining bytes from two disjoint writing systems). 4,292 candidate merges were filtered out by this rule; the remaining 2,156 were appended at the end of the merge list. The 216-entry gap between 2,372 surgery slots and 2,156 new merge rules accounts for word-level additions whose byte sequences are already reachable through existing merge chains in o200k_cropped (no new rule is required to compose them into a single token, only a new vocabulary entry at the composed sequence) plus atomic numeral and punctuation insertions (149 Indic numerals + 3 shared Brahmic punctuation tokens + a handful of broken-UTF-8 artifacts) that occupy single-byte or two-byte cells and likewise need no new BPE rule.

Why the entry count exceeds *vocab* – 256. The 301,398-entry count does not reflect a sequential BPE merge history. Because the artifact is converted from o200k_base’s `tiktoken` format to the HuggingFace BPE format, the inherited entries enumerate the valid binary decomposition paths for the 130,116 mergeable vocabulary tokens (about 2.3 paths per token on average) rather than a sequential record of greedy merges learned by frequency. The tokenizer’s `ignore_merges` flag is set to `true`, which gives whole-token vocabulary matches priority at encode time, so this path multiplicity does not affect tokenization. The standard sequential-BPE intuition ($merges \approx vocab - 256$) therefore does not apply to `tiktoken`-converted tokenizers; in this paper, “merge-rule entries” refers to entries in the HuggingFace merge list, not greedy-BPE merge operations.

3.7 Vocabulary ID ordering

Token IDs are assigned in descending order of corpus frequency, so that lower IDs correspond to more-frequent tokens. This ordering is behaviorally neutral: it changes neither the vocabulary, the merge rules, nor the output of `encode/decode` on any input. It is convenient for downstream training infrastructure that benefits from a frequency-contiguous vocabulary, including frequency-bucketed output layers (e.g., adaptive softmax variants (Grave et al., 2017)) and frequency-sorted data shards.

4 Internal validation

Before reporting external benchmarks against contemporary tokenizers (Section 5), we validate that the surgery preserved the properties of o200k_cropped that we wished to preserve, and that the new vocabulary functions correctly. This section reports five internal checks. All five are reviewer-runnable on a standard laptop.

4.1 English compression: BrahmicTokenizer-131K vs o200k_cropped

The first internal check is whether the surgery preserved English compression to the level we claimed. We tokenize FLORES-200 dev+devtest English (2,009 sentences, 51,712 words, 251,953 UTF-8 bytes) with both tokenizers and compare. BrahmicTokenizer-131K produces 63,919 tokens; o200k_cropped produces 63,941 tokens. The difference is 22 tokens across 2,009 sentences, a 0.034% reduction in total token count *in favor of* BrahmicTokenizer-131K. On per-sentence tokens, the two tokenizers agree on 1,944 of 2,009 sentences (96.8%) byte-identically.

The 22-token improvement is small enough to be attributable to the few char-infrastructure merges added by the surgery that happen to compose better with English token-boundary text, and small enough that the “bit-identical English” claim survives to 0.034% precision. We do not claim *strict* bit-identity; we claim *practical* preservation to within rounding. On bytes-per-token, BrahmicTokenizer-131K achieves 3.943 bytes/token on FLORES English versus o200k_cropped’s 3.942 bytes/token, a 0.025% difference also in BrahmicTokenizer-131K’s favor.

We additionally tested 38 common programming identifiers under both tokenizers (e.g., `springframework`, `javascript`, `includegraphics`, `requestAnimationFrame`, `getElementsByClassName`, plus Python keywords). All 38 produce byte-identical token sequences under BrahmicTokenizer-131K and o200k_cropped. The surgery touched no code-token-relevant merges.

4.2 Code and math compression

We tokenize three standard code/math evaluation corpora with both BrahmicTokenizer-131K and o200k_cropped: HumanEval (164 Python problems, prompts plus canonical solutions, 91,427 UTF-8 bytes total), MBPP-sanitized (257 problems, 268,194 bytes; the curated subset of MBPP, distinct from the 974-problem MBPP-full split used in Section 5.6), and GSM8K-train (7,473 problems, 8.0M bytes including chain-of-thought solutions). All three corpora produce identical-to-three-decimals compression: HumanEval 0.295 tokens/char, MBPP 0.296 tokens/char, GSM8K 0.301 tokens/char on both tokenizers. The bytes-per-token results are similarly identical: 3.385, 3.382, and 3.319 respectively on both tokenizers.

We additionally verified that BrahmicTokenizer-131K is in the strict-greedy 3-digit-grouping camp shared by o200k_base, GPT-OSS-120B, and Llama-3: the input ``1234567890'`` produces the token sequence ``123'` | ``456'' | ``789'' | ``0''` under all four tokenizers, byte-identical to o200k_cropped. The surgery’s merge-rule additions inserted at ranks 269,444 and beyond do not interfere with the digit-grouping merges that fire at much earlier ranks.

4.3 Round-trip integrity

A tokenizer is round-trip safe when `decode(encode(text)) == text` for arbitrary inputs. We tested round-trip integrity on a 1,000-sentence random sample drawn from the Indic SFT portion of the audit corpus, covering all 11 target Brahmic languages with proportional representation. All 1,000 sentences round-trip byte-perfect under BrahmicTokenizer-131K, including sentences containing mixed-script content (Latin + Brahmic), digits, punctuation, whitespace runs, and emoji.

Worked example. The Hindi word भारत (“India”, 4 characters, 12 UTF-8 bytes: E0 A4 AD E0 A4 BE E0 A4 B0 E0 A4 A4) encodes under BrahmicTokenizer-131K to a single token (vocabulary ID 66,526), which decodes back to the exact 12-byte sequence. Under Tekken/Sarvam-m, the same input encodes to two tokens (IDs 10342 and 93230, corresponding to the subword pieces भ and ारत), and also round-trips byte-perfect. The 1-vs-2 token-count difference reflects a single-token vocabulary entry for the high-frequency 4-character word भारत that BrahmicTokenizer-131K’s surgery added (token 66,526 is among the 1,443 word-level slots) and that Tekken/Sarvam-m’s vocabulary does not contain.

Higher-leverage example. The Hindi word विद्यालय (“school”, 8 characters, 24 UTF-8 bytes) similarly encodes to a single token under BrahmicTokenizer-131K (vocabulary ID 67,123) but to two tokens under Tekken/Sarvam-m. A high-frequency 8-character Brahmic word fitting in a single 131K-vocab token entry is the kind of compositional anchor the surgery’s word-level allocation was designed to provide; the bytes-per-token compression payoff on Indic web text accumulates from many such single-token-word instances.

The round-trip property is preserved by construction because the surgery added no new pre-tokenizer rules and modified no existing merge rules; round-trip is a property of the pre-tokenizer and decoder configuration, which are inherited unchanged from o200k_base. The 1,000-sentence sample test produces a SHA-256 hash of each input string and compares against the SHA-256 hash of `decode(encode(input))`; 1,000 of 1,000 hashes match across all 11 Brahmic languages.

4.4 The 23-test audit suite

We ran a 23-test internal audit suite covering tokenizer hygiene across vocabulary integrity, special-token handling, sequence-length distribution, multilingual coverage, edge cases, and adversarial input. The full test catalog is described in Appendix B; the summary results on BrahmicTokenizer-131K are 23 of 23 tests pass cleanly with no test failures. Selected results worth highlighting:

- **Test 11 (Edge Cases / Byte Fallback):** 13 hand-curated edge-case inputs all tokenize and decode cleanly, including empty strings, lone whitespace and newlines, null bytes, byte-order marks, 10,000-character ASCII runs, 100-emoji sequences, 50 zero-width-space insertions, multi-script mixed input, leading and trailing whitespace, and CRLF line endings. Zero unknown tokens emitted across all 13 cases.
- **Test 13 (Byte-Fallback Rate):** measured byte-fragment rate on the audit corpus. Per-language Indic rates fall in the 0.0–0.1% range; English rate is 0.1%; corpus-wide rate is 0.05%.
- **Test 17 (Adversarial Token Injection):** tested for special-token injection vulnerabilities via RTL-override (U+202E) and zero-width-space (U+200B) characters embedded near chat-template boundaries. The tokenizer does not detect or sanitize these characters in its tokenization step. This is a property shared by every

byte-level BPE tokenizer we tested; the test is documented as application-layer responsibility, not tokenizer responsibility.

- **Test 22 (Garbage Token Audit):** identifies 46 tokens (0.035% of vocab) flagged by the audit suite’s classifier rules: 20 with U+FFFD replacement characters (broken UTF-8 residue, excluding the legitimate 256-entry byte-fallback set), 18 containing zero-width or bidirectional control characters (U+200B, U+202A–E, U+FEFF, U+2060), 4 with Unicode private-use-area characters, and 4 that are HTML-entity artifacts (e.g. & #). All four classes are inherited from o200k_base’s BPE training on noisy web data; the surgery preserves them because they have nonzero fire counts on the audit corpus and removing them risks breaking inherited merge chains.

The 23-test audit suite was run on both BrahmicTokenizer-131K and o200k_cropped. The pass/fail signature is identical on both tokenizers (23/0/0). No test passes on o200k_cropped that fails on BrahmicTokenizer-131K.

4.5 Reviewer-runnable verification scripts

We ship four verification scripts with the artifact release, each runnable on a standard laptop in under 30 seconds:

- `verify_no_cross_script_merges.py`: scans all 301,398 merge-rule entries in the shipped `tokenizer.json` (see Section 3.6 for why this count exceeds `vocab - 256`) and confirms zero entries combine bytes from two disjoint writing systems.
- `verify_max_byte_length.py`: enumerates the 131,072 vocabulary tokens and confirms zero tokens exceed a configurable byte-length ceiling (default 32).
- `verify_sarvam_m_structural_identity.py`: confirms Sarvam-m’s BPE merge table and normal vocabulary are byte-identical to Tekken’s, with differences only in the reserved-special-token slots.
- `verify_kronecker_constraints_unified.py`: applies the byte-length and cross-script-token checks across multiple pre-tokenizer types (GPT-2 ByteLevel, SentencePiece Metaspace, plain Split, no pre-tokenizer), enabling apples-to-apples comparison across the 14-tokenizer set in Appendix A.

All four scripts are Apache 2.0-licensed and require only Python 3.9+ and the HuggingFace `tokenizers` library (Hugging Face, 2020). Reviewers can independently confirm every structural claim in the paper. The recommended smoke test:

```
1 BRAHMIC=/path/to/tokenizer.json
2 python verify_no_cross_script_merges.py $BRAHMIC && \
3 python verify_max_byte_length.py $BRAHMIC && \
4 python verify_kronecker_constraints_unified.py $BRAHMIC && \
5 python verify_sarvam_m_structural_identity.py $BRAHMIC $BRAHMIC
```

Listing 1: Smoke test for verification scripts.

Expected output: 4 PASS lines, exit 0.

5 External evaluation

This section reports BrahmicTokenizer-131K’s performance against contemporary tokenizers on five evaluation axes.

5.1 Token volume on 27M documents of public Indic pretraining text

Framing. The 27-million-document corpus and the 1.045-billion-token audit corpus used to score Stage-2 surgery candidates (Section 3) share their underlying sources: both are assembled from the same public AI4Bharat and Sarvam-AI releases. The 27M-corpus result is therefore an *in-distribution* measurement of BrahmicTokenizer-131K’s compression behavior. We report it as the headline figure because it is the closest available proxy for production training cost on an Indic-heavy mix, but the rank ordering and effect sizes deserve corroboration on corpora that were not visible to the surgery. Section 5.2 provides that corroboration on FLORES-200 and IN22-Gen, both of which are publicly downloadable, independent of our audit sources, and contain identical sentence sets across all 22 languages; we treat them as the out-of-distribution evidence.

The headline external-evaluation result concerns the most practically relevant metric for training-cost estimation: how many tokens does each tokenizer produce when run on a representative Indic pretraining corpus? We assembled a

27.12-million-document corpus from the public AI4Bharat Indic stack plus Sarvam-AI’s Samvaad-Hi conversational corpus and Sarvam-AI’s MMLU translation set. After filtering to Indic-language rows only, the corpus contains 2.84 billion whitespace-delimited words and 46.21 GB of UTF-8 text across 11 Brahmic-script languages. Per-language coverage ranges from 28 million words (Assamese) to 745 million words (Bengali).

We tokenized this corpus with both BrahmicTokenizer-131K and Tekken/Sarvam-m using the HuggingFace `tokenizers` library. Implementation details and reproduction instructions are in Appendix D. The headline result is in Table 3.

Table 3: Token-volume comparison on the 27M-document Indic pretraining corpus.

Language	Words	UTF-8 bytes	BrahmicTokenizer	Tekken/Sarvam-m	Advantage
Hindi	754 M	9.79 GB	1,231 M	1,480 M	−16.81%
Bengali	745 M	10.83 GB	1,638 M	1,974 M	−17.04%
Tamil	220 M	5.28 GB	684 M	812 M	−15.79%
Marathi	210 M	3.78 GB	529 M	657 M	−19.54%
Telugu	190 M	3.99 GB	599 M	742 M	−19.22%
Malayalam	155 M	4.10 GB	518 M	738 M	−29.81%
Kannada	103 M	2.19 GB	313 M	387 M	−19.11%
Punjabi	84 M	1.06 GB	210 M	258 M	−18.30%
Odia	60 M	1.00 GB	228 M	984 M	−76.79%
Gujarati	291 M	3.81 GB	605 M	910 M	−33.44%
Assamese	28 M	0.38 GB	67 M	100 M	−33.08%
TOTAL	2,841 M	46.21 GB	6,623 M	9,041 M	−26.75%

BrahmicTokenizer-131K produces 26.75% fewer tokens than Tekken/Sarvam-m on the same input text. In absolute terms, BrahmicTokenizer-131K produces 6.62 billion tokens versus Tekken/Sarvam-m’s 9.04 billion tokens for the same 2.84-billion-word corpus, a 2.42-billion-token reduction. The advantage holds on 11 of 11 languages with no exceptions, with per-language savings ranging from 15.79% (Tamil) to 76.79% (Odia). The Odia result is the most striking: BrahmicTokenizer-131K produces 228 million tokens against Tekken/Sarvam-m’s 984 million for the same 60 million words of input text, a $4.31\times$ ratio.

The per-language pattern reflects two structural realities documented in Section 5.4. **First**, Tekken/Sarvam-m contains zero Oriya-block tokens in its 131K vocabulary, so every Odia character falls to byte-level fallback at three tokens per Brahmic character. BrahmicTokenizer-131K’s surgery added 725 Oriya-block tokens to the vocabulary. **Second**, the languages where BrahmicTokenizer-131K’s advantage is smaller (Hindi, Bengali, Tamil at 15–17%) are languages where Tekken/Sarvam-m’s inherited o200k_base vocabulary already provides reasonable Devanagari, Bengali-script, and Tamil-script coverage.

For a training pipeline targeting Indic-heavy corpora, the 26.75% reduction translates substantially into per-step compute and context-length savings; the realized savings depend on architecture-specific factors (attention pattern, padding, optimizer state) but are first-order in the token-count reduction.

We caveat the corpus: the 27M-document corpus is Indic-only after filtering, so the figure represents the upper bound of the per-step compute advantage on Brahmic-heavy training mixes. For training pipelines with 30% or more non-Indic content, the savings on the non-Indic portion are zero (English compression is preserved bit-identically) and the aggregate savings dilute proportionally.

5.2 Per-word fertility on FLORES-200 and IN22-Gen

Section 5.1 reported token-volume compression on a curated 27-million-document corpus. This section corroborates those findings on two field-standard evaluation benchmarks where per-language sample sizes are matched: FLORES-200 dev+devtest (2,009 sentences per language) and IN22-Gen (1,024 sentences per language). Both benchmarks are publicly downloadable, ungated, and contain identical input sentences across all languages.

We evaluate 11 publicly available tokenizers on FLORES-200 and IN22-Gen. Three notable omissions from this fertility-ranked comparison: MUTANT-Indic (artifact not publicly available); GPT-4o’s tokenizer (unavailable, we use o200k_base instead); o200k_base / o200k_cropped / Airavata are excluded from the ranked fertility table to keep the rank-ordering focused on production multilingual tokenizers, but appear in the structural and bytes-per-token comparisons (Sections 5.3 and Appendix A). IndicBERTv2-SS is included with its measured English fertility (1.35); the tokenizer encodes English without UNK despite being trained for Indic-language classification.

Table 4 reports fertility on FLORES-200 across all 11 target Brahmic languages plus English as a reference anchor. The 11 tokenizers are ranked by mean fertility across the 11 Brahmic languages. Language abbreviations: Hi=Hindi, Bn=Bengali, Ta=Tamil, Te=Telugu, Kn=Kannada, Ml=Malayalam, Mr=Marathi, Gu=Gujarati, Pa=Punjabi, Or=Odia, As=Assamese.

Table 4: Per-word fertility on FLORES-200 dev+devtest. 11 tokenizers (rank-ordered), 11 Brahmic languages plus English. Lower is better.

Rank	Tokenizer	Vocab	En	Hi	Bn	Ta	Te	Kn	Ml	Mr	Gu	Pa	Or	As	Mean
1	Sarvam-30B	262K	1.24	1.39	1.68	2.36	2.32	2.56	2.94	2.00	1.93	1.63	1.97	2.82	2.14
2	Sarvam-1	68K	1.43	1.40	2.05	2.16	2.13	2.40	2.83	1.78	1.80	1.68	2.33	4.75	2.30
3	Gemma-3-1B	262K	1.24	1.39	1.68	2.36	2.83	3.26	3.33	2.00	2.40	2.82	4.83	2.82	2.70
4	BrahmicTokenizer-131K	131K	1.24	1.64	2.33	3.12	3.03	3.27	3.47	2.57	2.27	2.60	4.13	2.75	2.84
5	GPT-OSS-120B	200K	1.22	1.64	2.33	3.15	3.05	3.30	3.50	2.57	2.29	2.72	6.79	2.76	3.10
6	Tekken/Sarvam-m	131K	1.27	1.94	2.92	3.59	3.70	3.83	4.74	3.14	3.62	3.20	18.18	4.72	4.87
7	Krutrim-1	70K	1.33	3.30	4.62	6.73	5.68	6.12	7.35	4.94	4.02	3.24	4.85	4.49	5.03
8	DeepSeek-R1	129K	1.23	2.95	2.79	4.92	5.97	6.21	7.94	4.27	4.98	4.68	7.61	3.76	5.10
9	IndicBERTv2-SS	200K	1.35	1.25	5.04	8.10	6.62	6.69	8.67	1.47	4.99	3.91	5.58	5.23	5.23
10	Qwen3-8B	152K	1.26	4.75	7.10	10.00	11.35	11.84	13.29	6.70	8.86	7.76	13.60	7.70	9.36
11	Llama-3.1-8B	128K	1.24	2.67	7.99	12.28	13.23	14.90	16.16	3.99	9.91	8.19	16.78	8.59	10.43

BrahmicTokenizer-131K ranks 4 of 11 tokenizers with publicly downloadable artifacts on Brahmic mean fertility, sitting between Gemma-3-1B ($2\times$ vocab budget) and GPT-OSS-120B ($1.5\times$ vocab budget). At the 131K vocab class specifically, BrahmicTokenizer-131K is the best tokenizer in the comparison by a substantial margin: Tekken/Sarvam-m at 4.87 mean Brahmic fertility versus our 2.84 (a 41.8% relative improvement at the same vocab budget). The three tokenizers ranked above us (Sarvam-30B 262K, Sarvam-1 68K Indic-only, Gemma-3-1B 262K) are either at substantially larger vocabulary budgets or are Indic-specialist tokenizers.

Per-language extremums. On Odia, Tekken/Sarvam-m at 18.18 tokens/word, Llama-3.1-8B at 16.78, Qwen3-8B at 13.60, and DeepSeek-R1 at 7.61 all fall into byte-level fallback or near-fallback regimes. BrahmicTokenizer-131K at 4.13 tokens/word achieves a $4.40\times$ compression ratio over Tekken/Sarvam-m and remains the best Oriya-capable tokenizer at the 131K vocab class. On Assamese, BrahmicTokenizer-131K at 2.75 is competitive with Sarvam-30B (2.82) and better than Sarvam-1 (4.75).

IN22-Gen corroboration. Table 5 reports the same evaluation on IN22-Gen. The rank ordering is identical across the two benchmarks. BrahmicTokenizer-131K’s rank-4 position is robust to corpus variation; per-row IN22 means are 0.21–0.54 higher than FLORES, with the relative ordering between tokenizers preserved across both corpora.

Table 5: IN22-Gen mean Brahmic fertility ranking compared with FLORES-200.

Rank	Tokenizer	IN22-Gen mean	FLORES-200 mean
1	Sarvam-30B	2.35	2.14
2	Sarvam-1	2.57	2.30
3	Gemma-3-1B	2.92	2.70
4	BrahmicTokenizer-131K	3.08	2.84
5	GPT-OSS-120B	3.37	3.10
6	Tekken/Sarvam-m	5.21	4.87
7	Krutrim-1	5.27	5.03
8	DeepSeek-R1	5.43	5.10
9	IndicBERTv2-SS	5.50	5.23
10	Qwen3-8B	9.84	9.36
11	Llama-3.1-8B	10.97	10.43

Final note on the rank-4 position. Three tokenizers outrank BrahmicTokenizer-131K on FLORES-200 and IN22-Gen Brahmic mean fertility: Sarvam-30B (262K vocab, $2\times$ our budget), Sarvam-1 (68K vocab, vocabulary allocated heavily to Indic content at the cost of English, EU-language, and code compression efficiency, as a design choice), and Gemma-3-1B (262K vocab, $2\times$ our budget). At the 131K vocabulary class specifically, BrahmicTokenizer-131K is the best Brahmic-capable tokenizer in our comparison. The “general-purpose at 131K” qualifier matters; Section 5.5 quantifies the trade-offs that specialist or larger-vocab tokenizers make on non-Indic content.

5.3 Bytes-per-token compression

Bytes-per-token compression handles three cases per-word fertility handles poorly: languages without word boundaries, tokenizers with sub-word merges that depend on whitespace prefixes, and inputs where bytes-per-token captures memory and bandwidth cost more directly. Higher is better; the metric measures how much input data each token “carries.”

Table 6 reports bytes-per-token on a multi-domain composite corpus (FLORES-200 + sampled 27M corpus + HumanEval + MBPP + GSM8K), tokenized with 12 tokenizers.

Table 6: Bytes-per-token compression by corpus class. Higher is better.

Tokenizer	Vocab	English	Brahmic	Code	Math	All-corpus mean
Sarvam-30B	262K	4.99	1.91	3.21	3.07	3.30
BrahmicTokenizer-131K	131K	4.91	1.83	3.39	3.32	3.36
Sarvam-1	68K	4.20	2.20	2.64	2.62	2.91
Gemma-3-1B	262K	4.99	1.92	3.07	2.85	3.21
Tekken/Sarvam-m	131K	4.78	1.13	3.25	2.85	3.00
GPT-OSS-120B	200K	4.97	1.74	3.21	3.20	3.28
o200k_base	200K	4.97	1.74	3.21	3.20	3.28
DeepSeek-R1	129K	4.99	1.07	3.25	2.85	3.04
Krutrim-1	70K	4.10	1.18	2.81	2.63	2.68
Llama-3.1-8B	128K	4.99	0.55	3.27	2.85	2.91
Qwen3-8B	152K	4.98	0.61	3.28	2.85	2.93
Airavata	48K	3.80	1.74	2.42	2.31	2.57

BrahmicTokenizer-131K achieves the best all-corpus mean compression (3.36 bytes/token) of the 12 tokenizers. The advantage is balanced across the four corpus classes: English 4.91 (within 1.6% of Sarvam-30B’s leading 4.99), Brahmic 1.83 (rank 4 of 12, behind specialists), Code 3.39 (best in set, beating Tekken/Sarvam-m’s 3.25 by 4.3%), and Math 3.32 (best in set, beating o200k_base/GPT-OSS-120B’s 3.20 by 3.8%).

Pareto frontier at the 131K vocabulary class. BrahmicTokenizer-131K leads every other 131K-or-smaller tokenizer in the comparison on all-corpus mean. Specifically: vs Tekken/Sarvam-m (131K), BrahmicTokenizer-131K is 12.0% better; vs DeepSeek-R1 (129K), 10.5% better; vs Llama-3.1-8B (128K), 15.5% better; vs Krutrim-1 (70K), 25.4% better; vs Sarvam-1 (68K), 15.5% better overall despite losing on Brahmic.

The vocab-class boundary. Two tokenizers at 200K–262K vocab classes match or exceed BrahmicTokenizer-131K on certain individual axes: Gemma-3-1B (4.99 English, but loses on math/code), Sarvam-30B (4.99 English and wins on Brahmic by 4.4% but loses on code/math). Neither dominates BrahmicTokenizer-131K on all-corpus mean. No tokenizer leads BrahmicTokenizer-131K simultaneously on all four corpus classes at any vocabulary budget.

5.4 Structural comparison at the 131K vocabulary budget

This section compares the structural properties of BrahmicTokenizer-131K and Tekken/Sarvam-m at the same 131K vocabulary budget, explaining the per-language compression differences mechanistically. Table 7 reports five structural properties for the same-budget head-to-head pair.

Table 7: Structural properties at the 131K vocabulary budget.

Property	BrahmicTokenizer-131K	Tekken/Sarvam-m	Implication
Vocabulary size	131,072	131,072	identical budget
Tokens >32 UTF-8 bytes	0	56	structural property
Tokens spanning ≥ 2 scripts	0	8	structural purity
Oriya-block tokens	725	0	cause of $4.31\times$ Odia gap
Special/reserved tokens	12	18	minor; equivalent

The fourth row is the mechanistic explanation for Section 5.1’s most extreme per-language finding: Tekken/Sarvam-m contains zero Oriya-block tokens, so all Odia tokenization falls to byte-level fallback. BrahmicTokenizer-131K’s 725 Oriya tokens lift Odia from byte-fallback into the subword-piece regime. The “special/reserved tokens” row counts only the 12 structural special tokens that have distinct functional roles in the o200k_base inheritance chain (EOS, BOS, padding, FIM, multimodal markers); it is not the total `added_tokens` count in the shipped

`tokenizer.json`, which is 356 and includes 250 numbered reserved-slot placeholders (`<|reserved_0|>` through `<|reserved_249|>`) plus think-tag, response, and context markers. The 12-vs-18 comparison is on the functional subset only.

Table 8 expands the analysis to all 9 Brahmic scripts and decomposes the Stage-2 surgery’s 2,372-slot additions by content type.

Table 8: Stage-2 surgery additions by content type. The 2,372-slot budget breaks down into 2,215 single-script Brahmic additions, 149 Indic numeral merges, 3 shared Brahmic punctuation tokens, and 5 broken-UTF-8 artifacts.

Category	Slots
Single-script Brahmic additions	2,215
Oriya (Odia)	663
Gurmukhi (Punjabi)	328
Malayalam	225
Gujarati	203
Kannada	177
Devanagari (Hindi, Marathi)	173
Bengali (Bengali, Assamese)	155
Telugu	153
Tamil	138
Indic numerals (digit ranges across 9 scripts)	149
Shared Brahmic punctuation (danda, abbreviation signs)	3
Residual broken-UTF-8 artifacts	5
Total Stage-2 surgery additions	2,372

The LP-allocation policy prioritized scripts with the largest under-representation: Oriya at 663 (30% of single-script additions), Gurmukhi at 328, Malayalam at 225, Gujarati at 203. The two most-resourced scripts in o200k_base (Devanagari and Bengali) received only 173 and 155 single-script additions respectively.

Table 9 compares the per-script Brahmic-containing token counts at the same 131K vocabulary budget. At the same vocab budget, BrahmicTokenizer-131K contains $3.02\times$ more Brahmic-containing tokens than Tekken/Sarvam-m (15,709 vs 5,202).

Table 9: Per-script Brahmic-containing token counts at the 131K vocabulary budget. Ratio = BrahmicTokenizer-131K / Tekken/Sarvam-m.

Script	BrahmicTokenizer-131K	Tekken/Sarvam-m	Ratio
Devanagari (Hindi, Marathi)	4,165	1,569	$2.65\times$
Bengali (Bengali, Assamese)	2,292	839	$2.73\times$
Telugu	1,508	920	$1.64\times$
Kannada	1,497	570	$2.63\times$
Malayalam	1,908	406	$4.70\times$
Gujarati	1,832	204	$8.98\times$
Tamil	1,130	539	$2.10\times$
Oriya (Odia)	725	0	∞
Gurmukhi (Punjabi)	652	155	$4.21\times$
Total Brahmic-containing	15,709	5,202	$3.02\times$
Brahmic share of vocab	11.99%	3.97%	$3.02\times$

Why Tekken/Sarvam-m has zero Oriya tokens. Oriya text is significantly underrepresented in publicly available training corpora used by frequency-based BPE training, despite efforts like AI4Bharat’s Sangraha to collect Brahmic web content. Frequency-based BPE training systematically underproduces tokens for content classes below a certain frequency threshold, producing zero merges for sufficiently rare content. Sarvam-30B (262K vocab, Brahmic-heavy training set) does contain Oriya-block tokens because its training corpus deliberately upsampled Brahmic content. The BPE training methodology rewards frequency; the deliberate-vocabulary-design methodology (this paper’s approach) rewards comprehensiveness within a budget.

Synthesis across four tokenizers. Table 10 reports Brahmic-containing token counts across BrahmicTokenizer-131K, Tekken/Sarvam-m, Sarvam-30B, and Sarvam-1.

Table 10: Brahmic vocabulary share across four tokenizers.

Tokenizer	Vocab	Brahmic-containing	Brahmic share	Brahmic per 1K vocab
Tekken/Sarvam-m	131K	5,202	3.97%	39.7
BrahmicTokenizer-131K	131K	15,709	11.99%	119.9
Sarvam-30B	262K	51,474	19.66%	196.4
Sarvam-1	68K	49,707	73.28%	730.0

The Brahmic-per-1K-vocab gradient maps cleanly to design intent: Tekken/Sarvam-m at 39.7 per 1K reflects incidental Brahmic from frequency-based BPE training on multilingual web data; BrahmicTokenizer-131K at 119.9 per 1K (a $3\times$ increase) reflects deliberate Brahmic retrofit on a general-purpose base; Sarvam-30B at 196.4 per 1K reflects $2\times$ vocabulary budget with intentional Brahmic emphasis; Sarvam-1 at 730.0 per 1K reflects pure Brahmic specialization.

5.5 General-purpose capability versus specialist tokenizers

This section extends the comparison to specialist tokenizers at different vocabulary classes. The purpose is to verify the “general-purpose” framing of the paper empirically rather than rhetorically.

We evaluate on seven non-Indic axes: per-word fertility on FLORES-200 English, French, German, and Spanish; and per-character tokenization on HumanEval, MBPP-sanitized, and GSM8K-train. Results in Table 11.

Table 11: Cross-tokenizer comparison on non-Indic content. Lower is better. Bold marks the per-column winner.

Tokenizer	Vocab	En t/w	Fr t/w	De t/w	Es t/w	HumE t/c	MBPP t/c	GSM8K t/c
BrahmicTokenizer-131K	131K	1.235	1.464	1.653	1.388	0.295	0.320	0.301
Tekken/Sarvam-m	131K	1.267	1.432	1.659	1.405	0.307	0.338	0.351
Sarvam-1	68K	1.431	2.648	3.129	2.521	0.378	0.424	0.381
Sarvam-30B	262K	1.237	1.489	1.642	1.348	0.334	0.388	0.351

The vocab-controlled comparison. At the same 131K vocabulary budget, BrahmicTokenizer-131K beats Tekken/Sarvam-m on 5 of 7 non-Indic columns and ties on 1. The wins are English (-2.5%), HumanEval (-4.0%), MBPP (-5.4%), and GSM8K (-14.2%); the ties are German; the losses are French ($+2.2\%$) and Spanish ($+1.2\%$). Combined with the 26.7% Indic compression advantage from Section 5.1, BrahmicTokenizer-131K dominates Tekken/Sarvam-m across both Indic and non-Indic content at the same vocabulary budget.

The Indic-specialist comparison: Sarvam-1. Sarvam-1 allocates 73% of its 68K vocabulary to Indic content. The cost on non-Indic is severe: 15.9% worse on English than BrahmicTokenizer-131K, 81–89% worse on French/German/Spanish, and 26–33% worse on the three code/math corpora. The trade-off is symmetric: per Section 5.1, Sarvam-1 produces 12.7% fewer tokens on Brahmic content than BrahmicTokenizer-131K.

The $2\times$ -vocab comparison: Sarvam-30B. Sarvam-30B has 262K vocabulary slots, double BrahmicTokenizer-131K’s budget. The expected advantage on non-Indic given the resource asymmetry is significant; the observed result is more nuanced: Sarvam-30B ties or narrowly beats us on EU languages and English, but *loses* to us on all three code/math corpora by margins of 13.2–21.3%. Despite $2\times$ vocabulary budget, Sarvam-30B’s code/math compression is materially worse than BrahmicTokenizer-131K’s because its vocabulary allocation prioritized Brahmic coverage and squeezed code/math representation.

Synthesis. Among the four tokenizers, only BrahmicTokenizer-131K achieves simultaneously: (a) o200k-grade English compression (1.235 tokens/word, within 0.5% of o200k_base’s 1.23), (b) best-in-set code compression on HumanEval and MBPP, (c) best-in-set math compression on GSM8K with a 14.2% advantage over the next-best, (d) competitive EU language compression within 3% of best on French/German/Spanish, and (e) substantial Brahmic compression that beats Tekken/Sarvam-m by 26.7% on real Indic pretraining text. This is the empirical content of the general-purpose claim.

5.6 Code and math compression mechanism

Sections 5.2, 5.3, and 5.5 reported that BrahmicTokenizer-131K achieves best-in-class code and math compression among the publicly available tokenizers in our 14-tokenizer benchmark. This section explains the mechanism: the wins come from preservation of o200k_base’s tokenization choices, not from new tokenizer features.

Individual-digit tokenization on numeric inputs. Mathematical reasoning is heavily affected by how a tokenizer handles numbers. BrahmicTokenizer-131K inherits o200k_base’s 3-digit grouping behavior bit-identically:

```
1 from transformers import AutoTokenizer
2 tokenizer = AutoTokenizer.from_pretrained('theschoolofai/BrahmicTokenizer-131K')
3 tokenizer.encode("1234567890", add_special_tokens=False)
4 # -> [4660, 14932, 23133, 26]
5 # Decoded: ['123', '456', '789', '0']
```

Listing 2: Digit-grouping behavior on a 10-digit number.

GSM8K’s chain-of-thought reasoning involves frequent arithmetic. The 3-digit grouping strategy produces approximately 33% fewer tokens for typical numeric strings than alternative strategies (which translates to the 14.2% advantage we measured on GSM8K-train). We did not engineer this advantage; we preserved it.

Programming identifier tokenization. We tested 38 common programming identifiers under both BrahmicTokenizer-131K and o200k_cropped:

```
1 sample_identifiers = ['springframework', 'javascript', 'includegraphics',
2                       'requestAnimationFrame', 'getElementsByClassName',
3                       'BufferedReader', 'PrintWriter', 'StringBuilder']
4 for ident in sample_identifiers:
5     brahmic_tokens = brahmic_tokenizer.encode(ident, add_special_tokens=False)
6     o200k_tokens = o200k_tokenizer.encode(ident, add_special_tokens=False)
7     assert brahmic_tokens == o200k_tokens, f"differ on {ident}"
```

Listing 3: Identifier byte-identity check.

All 38 identifiers produce byte-identical token sequences. The surgery touched no code-token-relevant merges. This identical-token-sequence preservation extends across the HumanEval and MBPP corpora: per-document tokenization is byte-identical on 100% of the 164 HumanEval problems and 100% of the 974 problems in MBPP-full (the unfiltered split that supersedes the 257-problem MBPP-sanitized split used in Section 4).

Why the wins against larger-budget tokenizers. BrahmicTokenizer-131K’s vocabulary is largely o200k_base-derived (88% of slots), which means its code and math compression matches o200k_base’s strong baseline. Sarvam-30B’s 262K slots are heavily devoted to Brahmic content (51,474 Brahmic-containing tokens), leaving fewer slots for code identifiers and English/math vocabulary. The gap is structural and reflects vocabulary allocation choices.

6 Ablations and analyses

6.1 Per-script LP allocation policy

To validate the LP-optimal allocation against reasonable alternatives, we ran four allocation policies and measured the audit-corpus token savings each would have produced. The ablation uses simulation: for each policy, we score against the per-script saturation curves the surgery produced, isolating the allocation-policy effect from the candidate-pool effect.

Table 12: Audit-corpus token savings for five allocation policies at the 2,215-slot single-Brahmic-script budget.

Policy	Slots used	Total tokens saved	Δ vs shipped
Shipped (heuristic)	2,215	6,949,473	baseline
LP-optimal (greedy)	2,215	6,949,473	+0.00%
Worst-script-first	2,215	6,947,436	−0.03%
Frequency-only	1,786	6,769,060	−2.60%
Equal per script	1,827	6,604,464	−4.96%

The LP-optimal greedy solver and the shipped heuristic reach identical total savings ($\Delta = 0.00\%$), although their per-script allocations differ slightly: the optimum is flat over a small family of equal-total allocations, so multiple

allocations achieve the maximum. The shipped heuristic was empirically optimal at the 2,215-slot budget. Worst-script-first captures 99.97% of the heuristic’s gains despite ignoring saturation curve shape; the single source of loss is Devanagari short-allocation. Frequency-only loses 2.60% by over-allocating to high-resource scripts. Equal allocation is the empirical floor at -4.96% .

The most important finding is the non-result: the shipped heuristic sits in a flat optimum. Any allocation policy that biases toward the under-resourced scripts achieves $\geq 99.97\%$ of the maximum savings. The specific per-script ratios are not load-bearing; the shape of biasing toward under-resourced scripts is.

6.2 Dead-slot histogram and frequency-floor policy

The Stage-2 surgery removed 2,372 corpus-dead tokens from o200k_cropped. Of o200k_cropped’s 131,072 tokens, 128,700 fire at least once on the audit corpus (a 98.2% utilization rate). Of the remaining 2,372 corpus-dead tokens, 2,083 fire zero times across the entire 1.045-billion-token audit, and the 289 marginal cases (1–1,000 fires across 1B tokens) are tokens for out-of-scope content classes that we chose to remove by policy. The signal-to-noise ratio between the median kept token and the median dropped token is approximately $197,000\times$; the ratio between the lowest-firing kept token and the highest-firing dropped token is approximately $12\times$.

6.3 No-cross-script-merge rule

Section 3.6’s surgery introduced a no-cross-script-merge rule preventing new merges from combining bytes from two disjoint writing systems. We did not ablate “what if we’d allowed cross-script merges?” because the rule was set by architectural compatibility considerations. A hypothetical ablation: if we’d allowed cross-script merges, the surgery could potentially have added 3–5% additional merge entries (since some Brahmic-Latin boundary patterns are frequent on bilingual content). The cost would be loss of the cross-script purity property.

6.4 New-merge utilization on the audit corpus

The Stage-2 surgery added 2,156 new merges. We measured how often each fires on the audit corpus:

Table 13: New-merge utilization distribution on the audit corpus.

Fire rate	New merge count	% of new merges
0 fires (unfired)	569	26.4%
1–1,000 fires	312	14.5%
1,001–100,000 fires	681	31.6%
100,001–10,000,000 fires	487	22.6%
10,000,001+ fires	107	5.0%
Total	2,156	100%

The 569 unfired merges represent a 26.4% “wasted budget” rate. Two interpretations are reasonable: (a) the unfired merges are vocabulary for under-represented content classes that will fire on production data, or (b) they are dead vocabulary that could have been better allocated.

6.5 Design choices not ablated

Three design choices were made on engineering judgment without empirical ablation: the 2,372-slot count (determined by dead-slot supply), the pre-tokenizer choice (inherited from o200k_base for drop-in compatibility), and the audit corpus composition (best estimate of production training distribution).

7 Limitations and future work

7.1 Brahmic specialization gap at other vocabulary budgets

Section 5.5 reports that specialist tokenizers at vocabulary budgets other than 131K outperform BrahmicTokenizer-131K on per-language Brahmic compression. Sarvam-30B at 262K produces 18.7% fewer tokens than BrahmicTokenizer-131K on the 27M-document Indic pretraining corpus. Sarvam-1 at 68K produces 12.7% fewer tokens. MUTANT-Indic’s published per-language numbers (Rana et al., 2026) suggest similar or stronger Indic-specialist

performance. These losses are not failures of the construction; they are intentional trade-offs of the general-purpose-at-131K design.

7.2 The 88% non-Brahmic vocabulary was not audited

The Stage-2 surgery modified 2,372 of 131,072 vocabulary slots (1.81%). The remaining 128,700 slots were inherited from o200k_cropped. Any latent issues in o200k_base’s vocabulary propagate to BrahmicTokenizer-131K. One such issue was flagged in Section 4.4: 46 tokens classified as “garbage” by the audit suite (20 broken-UTF-8 residues outside the byte-fallback set, 18 zero-width/bidi control sequences, 4 private-use-area tokens, 4 HTML-entity artifacts). All four classes are inherited from o200k_base and have nonzero corpus fire rates; together they occupy 0.035% of the vocabulary.

7.3 Audit corpus representativeness

The 1.045-billion-token audit corpus used in Section 3 and the 27-million-document evaluation corpus of Section 5.1 are drawn from the same AI4Bharat and Sarvam-AI source releases. The 27M-corpus result is therefore an in-distribution measurement, not an independent test. The FLORES-200 and IN22-Gen fertility results in Section 5.2 are the out-of-distribution evidence: both benchmarks are publicly curated, independent of our audit sources, and the rank ordering and effect sizes on those benchmarks are consistent with the in-distribution 27M-corpus result. Production training data may still have different per-language distributions from any of the above. The 569 new merges that did not fire on the audit corpus represent 26.4% of the surgery’s new vocabulary entries; we cannot determine without further measurement whether they are useful in production or dead vocabulary.

7.4 Future directions

Several directions could extend this work:

Additional whole-word vocabulary entries. Per-language compression advantages on Hindi, Bengali, Tamil, and Punjabi sit in a low band (approximately 15–18% on the 27M corpus, Table 3), while Oriya, Gujarati, and Assamese sit in a high band (33–77%), with Malayalam at ~30% as a transitional case. Adding high-frequency whole-word entries for the low-band languages could close part of this gap, but at the cost of vocabulary budget. The 569 unfired new merges and the 5 broken-UTF-8 artifacts are candidate sources for reallocation if a future audit confirms they remain unfired.

EU language compression improvement. French, German, and Spanish fertilities are within 2–3% of best in our comparison. Targeted improvement would require additional analysis of o200k_base’s existing coverage.

Indic numeral expansion. The 149 Indic numeral merges added in v1 cover basic Brahmic digit tokens but not multi-digit Brahmic number sequences.

MUTANT-Indic comparative analysis. If the MUTANT-Indic artifact is released publicly, future work could benchmark it on our evaluation corpora.

Domain-specific tokenization studies. The methodology (audit, dead-slot identification, surgical retrofit) is applicable to domain-specific construction.

8 Conclusion

BrahmicTokenizer-131K is a 131,072-vocabulary byte-level BPE tokenizer that closes the Brahmic compression gap at the 131K vocabulary class while preserving o200k_base’s English, EU language, and code compression. We constructed it through a two-stage retrofit: a script-prune crop reducing 200,019 tokens to 131,072, and a surgical Brahmic retrofit of 2,372 corpus-dead slots determined by linear-programming allocation.

On 27 million documents of Indic pretraining text (2.84 billion words, 46.21 GB), BrahmicTokenizer-131K produces 26.7% fewer tokens than Tekken/Sarvam-m at the same vocabulary budget, with per-language savings ranging from 15.79% (Tamil) to 76.79% (Odia, a $4.31\times$ ratio). On English, EU language, code, and math evaluation, BrahmicTokenizer-131K matches or beats Tekken/Sarvam-m by 2.5–14.2% across the seven non-Indic axes we measured. Across our 14-tokenizer benchmark, BrahmicTokenizer-131K is the only tokenizer simultaneously competitive on every evaluation axis at the 131K vocabulary budget.

Two structural observations are worth surfacing. First, the $4.31\times$ Odia compression difference between BrahmicTokenizer-131K and Tekken/Sarvam-m is mechanistically explained by Tekken/Sarvam-m containing zero

Oriya-block tokens; the Stage-2 surgery’s 725 Oriya additions lift Odia from byte-level fallback to subword tokenization. Second, the 12% Brahmic vocabulary share in BrahmicTokenizer-131K (versus 4% in Tekken/Sarvam-m, 19.7% in Sarvam-30B, 73% in Sarvam-1) is the quantitative shape of the general-purpose-vs-specialist trade-off.

We release BrahmicTokenizer-131K under Apache 2.0 license at <https://huggingface.co/theschoolofai/BrahmicTokenizer-131K> and <https://github.com/theschoolofai/BrahmicTokenizer-131K>. Training pipelines currently using o200k_base can swap in BrahmicTokenizer-131K’s `tokenizer.json` without modifying data-loader, pre-tokenizer, or decoder code; the embedding matrix resizes from 200,019 to 131,072 rows as in any standard vocabulary switch. The vocabulary content differs in 40,717 of 131,072 slots, but the tokenizer-side interface is identical.

The methodology of this paper, surgical vocabulary retrofit of a well-engineered source tokenizer, is applicable to other situations where a high-quality source tokenizer exists for a target subset of languages and the target deployment requires additional language coverage. The construction is substantially more compute-efficient than training a tokenizer from scratch on a Brahmic-heavy corpus, while preserving all source-tokenizer properties by construction. We hope the methodology proves useful for future tokenizer construction.

Acknowledgments

This work was carried out at The School of AI, India. Compute resources were provided by AWS Cloud (spot-instance VMs). The 27-million-document Indic pretraining corpus was assembled from publicly available datasets released by AI4Bharat (Sangraha, Bharat Parallel Corpus Collection, Samanantar, NLLB-filtered, IndicComparable, ILCI) and Sarvam AI (Samvaad-Hi, MMLU translation set). We thank the ERA V4 cohort at The School of AI for their feedback and verification of preliminary results.

The verification scripts use the HuggingFace `tokenizers` library and OpenAI’s `tiktoken` library. Tokenizer comparisons reference public model releases from Mistral AI, Sarvam AI, Google DeepMind, Meta AI, Alibaba DAMO Academy, DeepSeek AI, OpenAI, Krutrim AI Labs, and AI4Bharat.

References

- O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. Mortensen, N. A. Smith, and Y. Tsvetkov. Do all languages cost the same? Tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL <https://arxiv.org/abs/2305.13707>. arXiv:2305.13707. Documents per-language pricing/latency costs induced by tokenizer fertility differentials.
- J. Austin, A. Odena, M. Nye, et al. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>. MBPP-sanitized.
- A. Bendale, M. Sapienza, S. Ripplinger, S. Gibbs, J. Lee, and P. Mistry. SUTRA: Scalable multilingual language model architecture, 2024. URL <https://arxiv.org/abs/2405.06694>. arXiv:2405.06694. Multilingual LLM architecture cited as comparison baseline by MUTANT (Rana et al., 2026).
- M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>. HumanEval: 164 Python problems.
- K. Cobbe, V. Kosaraju, M. Bavarian, et al. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>. GSM8K: grade-school math word problems.
- M. R. Costa-jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Heffernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard, et al. No language left behind: Scaling human-centered machine translation, 2022. URL <https://arxiv.org/abs/2207.04672>. arXiv:2207.04672. Releases the FLORES-200 dev+devtest benchmark (2009 sentences per language). A later version of this work appeared as Costa-jussà et al., *Nature* 630:841–846, 2024.
- Y. Cui, Z. Yang, and X. Yao. Efficient and effective text encoding for Chinese LLaMA and Alpaca, 2023. URL <https://arxiv.org/abs/2304.08177>. arXiv:2304.08177. Vocabulary expansion of LLaMA for Chinese; closest prior precedent for adding script coverage to an existing tokenizer via vocabulary surgery rather than from-scratch training.
- DeepSeek AI. DeepSeek-R1 model artifacts, 2025. URL <https://huggingface.co/deepseek-ai/DeepSeek-R1>. HF model card. Tokenizer vocab size 128,815. See DeepSeek-AI (2025) for the technical report.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL <https://arxiv.org/abs/2501.12948>.

- S. Doddapaneni, M. S. U. R. Khan, D. Verma, A. Vasanthkumar, A. Lavania, A. Kunchukuttan, M. M. Khapra, and R. Dabre. Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023. URL <https://arxiv.org/abs/2212.05409>. arXiv:2212.05409. Releases IndicBERTv2 / IndicNLG-Suite.
- P. Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.
- J. Gala, P. A. Chitale, A. K. Raghavan, V. Gumma, S. Doddapaneni, A. Kumar M, J. A. Nawale, A. Sujatha, R. Pudupully, V. Raghavan, P. Kumar, M. M. Khapra, R. Dabre, and A. Kunchukuttan. IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages. *Transactions on Machine Learning Research (TMLR)*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=vft4YuzAYA>. arXiv:2305.16307. Introduces the IN22 benchmark suite including IN22-Gen.
- J. Gala, T. Jayakumar, J. A. Husain, A. Kumar M, M. S. U. R. Khan, D. Kanojia, R. Pudupully, M. M. Khapra, R. Dabre, R. Murthy, and A. Kunchukuttan. Airavata: Introducing Hindi instruction-tuned LLM. *arXiv preprint arXiv:2401.15006*, 2024. URL <https://arxiv.org/abs/2401.15006>.
- Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Google DeepMind. Gemma 3 model artifacts, 2025. URL <https://huggingface.co/google/gemma-3-1b-pt>. HF model card. Tokenizer vocab size 262,145. See Gemma Team (2025) for the technical report.
- A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>. Llama 3.1 family; tokenizer vocab size 128,256.
- É. Grave, A. Joulin, M. Cissé, D. Grangier, and H. Jégou. Efficient softmax approximation for GPUs. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. URL <https://arxiv.org/abs/1609.04309>. arXiv:1609.04309. Adaptive softmax: frequency-bucketed output layers for large-vocabulary language models.
- Hugging Face. HuggingFace tokenizers library, 2020. URL <https://github.com/huggingface/tokenizers>. Rust-backed fast tokenization library.
- G. N. Jha. The TDIL program and the Indian language corpora initiative (ILCI). In *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC)*. European Language Resources Association (ELRA), 2010. URL <http://www.lrec-conf.org/proceedings/lrec2010/summaries/874.html>.
- M. S. U. R. Khan, P. Mehta, A. Sankar, U. Kumaravelan, S. Doddapaneni, S. B, V. B. G, S. Jain, A. Kunchukuttan, P. Kumar, R. Dabre, and M. M. Khapra. IndicLLMSuite: A blueprint for creating pre-training and fine-tuning datasets for Indian languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15831–15879, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.acl-long.843. URL <https://aclanthology.org/2024.acl-long.843/>. arXiv:2403.06350. ACL 2024 Outstanding Paper. Releases the Sangraha pre-training corpus (251B tokens, 22 Indic languages) and IndicAlign.
- Krutrim AI Labs. Krutrim-1-instruct: A multilingual base model and tokenizer for Indian languages, 2024. URL <https://huggingface.co/krutrim-ai-labs/Krutrim-1-instruct>. Vocab size 70,212, Indic-focused. Tokenization details discussed in arXiv:2407.12481 (Kallappa et al., “Krutrim LLM: Multilingual Foundational Model for over a Billion People”, 2024).
- A. Liu, J. Hayase, V. Hofmann, S. Oh, N. A. Smith, and Y. Choi. SuperBPE: Space travel for language models. In *Proceedings of the Second Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.13423>. arXiv:2503.13423. Multi-word subword vocabulary learning.
- Meta AI. Llama 3.1 model artifacts, 2024. URL <https://huggingface.co/meta-llama/Llama-3.1-8B>. HF model card for the Llama 3.1 8B base model. See Grattafiori et al. (2024) for the technical report.
- B. Minixhofer, F. Paischer, and N. Rekabsaz. WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2022. URL <https://arxiv.org/abs/2112.06598>. arXiv:2112.06598. Seminal work on transferring a pretrained model to a new tokenizer/language via embedding initialization.

- Mistral AI. Mistral-Nemo-Base-2407: A 12B-parameter base model with the Tekken tokenizer, 2024. URL <https://huggingface.co/mistralai/Mistral-Nemo-Base-2407>. Released July 2024. Tokenizer: Tekken, vocab size 131,072.
- OpenAI. o200k_base: Open-source tokenizer used in OpenAI’s GPT-4o, 2024. URL https://github.com/openai/tiktoken/blob/main/tiktoken_ext/openai_public.py. Released May 2024 with gpt-4o-2024-05-13. Vocab size 200,019.
- OpenAI. GPT-OSS-120B: An open-weights language model from OpenAI, 2025. URL <https://huggingface.co/openai/gpt-oss-120b>. Tokenizer vocab size 200,019; structurally identical to o200k_base.
- A. Petrov, E. La Malfa, P. H. S. Torr, and A. Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2305.15425>. arXiv:2305.15425. Quantifies cross-language tokenization disparities for closed and open tokenizers; provides the framing for “tokenization tax” in low-resource languages.
- Qwen Team. Qwen3 model artifacts, 2025. URL <https://huggingface.co/Qwen/Qwen3-8B>. HF model card. Tokenizer vocab size 151,669. See Yang et al. (2025) for the technical report.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. GPT-2; introduces byte-level BPE with the GPT-2 ByteLevel pre-tokenizer.
- G. Ramesh, S. Doddapaneni, A. Bheemaraj, M. Jobanputra, R. AK, A. Sharma, S. Sahoo, H. Diddee, M. J. D. Kakwani, N. Kumar, A. Pradeep, S. Nagaraj, K. Deepak, V. Raghavan, A. Kunchukuttan, P. Kumar, and M. S. Khapra. Samanantar: The largest publicly available parallel corpora collection for 11 Indic languages. *Transactions of the Association for Computational Linguistics (TACL)*, 10:145–162, 2022. URL <https://aclanthology.org/2022.tacl-1.9/>. arXiv:2104.05596.
- S. Rana, A. Menezes, A. Kulkarni, C. Khatri, and S. Agarwal. MUTANT: A recipe for multilingual tokenizer design. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2026. URL <https://arxiv.org/abs/2511.03237>. arXiv:2511.03237v2. Krutrim AI Labs, Bangalore. Acceptance attested by authors at github.com/ola-krutrim/MUTANT. MUTANT-Indic tokenizer artifact not publicly available at our paper preparation date.
- Sarvam AI. Sarvam-1: 2B-parameter base model for Indian languages, 2024a. URL <https://huggingface.co/sarvamai/sarvam-1>. Released October 2024. Tokenizer vocab size 68,096, Indic-focused.
- Sarvam AI. Samvaad-Hi v1: Multi-turn conversations in English, Hindi, and Hinglish with Indic context. Hugging Face dataset, 2024b. URL <https://huggingface.co/datasets/sarvamai/samvaad-hi-v1>. 100k high-quality multi-turn conversations. Released February 2024.
- Sarvam AI. Sarvam-30B: 30B-parameter base model with extended Brahmic vocabulary, 2025a. URL <https://huggingface.co/sarvamai/sarvam-30b>. Apache 2.0; 32B parameters, 23 languages; 19 layers, 128 experts with top-6 routing, 2.4B active parameters. Tokenizer vocab size 262,144.
- Sarvam AI. Sarvam-m: 24B-parameter base model using the Tekken tokenizer, 2025b. URL <https://huggingface.co/sarvamai/sarvam-m>. Tokenizer vocab size 131,072; structurally identical to Mistral-Nemo Tekken per our 14-tokenizer benchmark.
- R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *Proceedings of the ACL*, 2016.
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.

A Structural Diagnostics

A.1 Cross-script tokens and byte-length ceilings: 14 tokenizers

Table 14 reports structural properties for all 14 tokenizers in our benchmark, measured by each tokenizer’s own decoder. At a 32-byte ceiling and zero cross-script tokens, only BrahmicTokenizer-131K and o200k_cropped satisfy both constraints.

Table 14: Structural diagnostic across 14 publicly available tokenizers.

#	Tokenizer	Pre-tokenizer	Vocab	Max byte	Tokens >32 B	Cross-script	Both clean?
1	BrahmicTokenizer-131K (ours)	ByteLevel	131,072	32	0	0	Yes
2	o200k_cropped (pre-surgery)	ByteLevel	131,072	32	0	0	Yes
3	o200k_base (200K)	tiktoken BPE	200,019	128	266	59	No
4	Tekken (Mistral-Nemo, 131K)	ByteLevel	131,072	76	56	8	No
5	Sarvam-m (131K)	ByteLevel	131,072	76	56	8	No
6	GPT-OSS-120B (200K)	ByteLevel	200,019	128	266	59	No
7	Sarvam-1 (68K)	Metaspace	68,096	36	559	0	No
8	Sarvam-30B (262K)	Split	262,144	63	616	119	No
9	Gemma-3-1B (262K)	Split	262,145	48	379	0	No
10	Krutrim-1-instruct (70K)	ByteLevel	70,212	128	7	154	No
11	Airavata (48K)	None	48,065	48	277	0	No
12	Llama-3.1-8B (128K)	ByteLevel	128,256	128	231	7	No
13	Qwen3-8B (152K)	ByteLevel	151,669	128	240	0	No
14	DeepSeek-R1 (129K)	ByteLevel	128,815	128	769	11	No

A.2 Stage-1 incidental removal of constraint-violating tokens

Of o200k_base’s 266 over-byte tokens and 59 cross-script tokens, every one was removed by the Stage-1 script-prune crop. The cross-script tokens were predominantly CJK+Latin (57 of 59) and were removed as a side-effect of the foreign-script filter; the over-byte tokens were predominantly Thai (45) and structural fillers (185). This validates the claim in Section 3.2 that the structural properties are inherited from the script-prune crop rather than engineered by the surgery.

A.3 Verification script outputs

The four verification scripts produce the following expected output on BrahmicTokenizer-131K:

```
$ python verify_no_cross_script_merges.py FINAL_TOKENIZER/tokenizer.json
tokenizer:          FINAL_TOKENIZER/tokenizer.json
merge-rule entries: 301,398
cross-script:      0
PASS: no cross-script entries in the merge list.
```

```
$ python verify_max_byte_length.py FINAL_TOKENIZER/tokenizer.json
tokenizer:          FINAL_TOKENIZER/tokenizer.json
vocab size:        131,072
max_bytes limit:   32
tokens > 32 bytes: 0
PASS: all 130,716 normal tokens are within 32 bytes.
```

All four scripts return exit code 0 on PASS.

B Audit Suite Details

B.1 23-test catalog

The 23-test audit suite covers tokenizer hygiene across six categories:

Tests 1–5: Vocabulary integrity. Vocabulary count matches declaration; no duplicate token IDs; all vocabulary entries are valid UTF-8; no control characters except documented whitespace; pre-tokenizer configuration matches the o200k_base reference.

Tests 6–10: Special token handling. All 356 special tokens have unique IDs and surface forms; special tokens decode to expected literals; no special-token substring overlap with normal tokens; the EOS token (<|end_of_text|>, ID 36), BOS token (<|begin_of_text|>, ID 130,725), and padding token (<|pad|>, ID 130,726) occupy their post-permutation IDs (Section 3.7); the rest of the special-token block is inherited from o200k_base’s reserved-token range and includes the standard FIM (<|fim_prefix|>, <|fim_middle|>, <|fim_suffix|>), multimodal

(<|vision_start|>, <|image_pad|>, <|video_pad|>), and 250 numbered reserved slots, all at high IDs.

Tests 11–13: Edge cases and byte fallback. 13 edge-case inputs all tokenize cleanly; round-trip integrity on 1,000 sentences (1,000/1,000 byte-perfect); per-language byte-fragment rate 0.0–0.1% on Indic.

Tests 14–18: Multilingual coverage. Per-script vocabulary coverage matches Table 8; no Indic-language byte-fragment rate exceeds 0.5%; whitespace tokenization preserves GPT-2 ByteLevel “ $\dot{G}$ ” convention; adversarial token injection (U+202E, U+200B) documented as application-layer responsibility; sequence-length distribution reasonable.

Tests 19–23: Configuration and miscellaneous. Tokenizer JSON parses cleanly; loadable via `Tokenizer.from_file()` and `AutoTokenizer.from_pretrained()`; Apache 2.0 license present; garbage-token audit (46 tokens inherited from o200k_base: 20 broken-UTF-8, 18 zero-width/bidi, 4 PUA, 4 HTML-entity); EOS/BOS termination consistent with HuggingFace defaults.

B.2 Dead-slot histogram

The per-token fire-rate distribution on the audit corpus is bimodal: 128,700 tokens fire at rates $\geq 1,000$ per billion-token audit (the kept tokens), and 2,372 tokens fire at rates $\leq 1,000$ per billion (the dropped/replaced tokens). The boundary is set by policy: tokens for out-of-scope content (CJK, Arabic, etc.) were removed even if they fired non-zero times. The signal-to-noise ratio between median kept and median dropped tokens is approximately $197,000\times$.

C 4-way Indic per-language detail

C.1 Per-source corpus accounting

The 27M-document Indic pretraining corpus was assembled from the AI4Bharat Sangraha monolingual subsets plus parallel/conversational corpora. Per-language word counts range from 28 million (Assamese) to 745 million (Bengali).

C.2 4-way 27M-document comparison

Table 15 reports per-language token counts for all four tokenizers (BrahmicTokenizer-131K, Tekken/Sarvam-m, Sarvam-1, Sarvam-30B) on the 27M-document Indic pretraining corpus. The vocab-controlled comparison (BrahmicTokenizer-131K vs Tekken/Sarvam-m) is in Section 5.1; the cross-vocab comparisons surface here.

Table 15: 4-way 27M-document token counts. Negative percentages indicate the competitor uses fewer tokens than BrahmicTokenizer-131K (competitor wins); positive percentages indicate the competitor uses more tokens (BrahmicTokenizer-131K wins).

Language	Brahmic	Tekken/Sm Δ	Sarvam-30B	Δ	Sarvam-1	Δ
Assamese	67 M	+49.4%	68 M	+1.9%	100 M	+49.6%
Bengali	1,638 M	+20.5%	1,282 M	−21.7%	1,524 M	−6.9%
Gujarati	605 M	+50.2%	551 M	−8.9%	561 M	−7.2%
Hindi	1,231 M	+20.2%	1,053 M	−14.5%	1,116 M	−9.4%
Kannada	313 M	+23.6%	262 M	−16.2%	261 M	−16.7%
Malayalam	518 M	+42.5%	463 M	−10.6%	492 M	−5.0%
Marathi	529 M	+24.3%	414 M	−21.7%	408 M	−22.9%
Odia	228 M	+330.8%	118 M	−48.5%	144 M	−36.8%
Punjabi	210 M	+22.4%	138 M	−34.5%	139 M	−34.0%
Tamil	684 M	+18.8%	536 M	−21.7%	546 M	−20.2%
Telugu	599 M	+23.8%	500 M	−16.6%	491 M	−18.1%
TOTAL	6,623 M	+36.5%	5,385 M	−18.7%	5,782 M	−12.7%

Notable patterns: BrahmicTokenizer-131K beats all three competitors on Assamese. Sarvam-30B leads on 10 of 11 languages but loses to Sarvam-1 on Kannada and Telugu. The Odia row dominates the Sarvam-m gap (+330.8%); excluding Odia, Tekken/Sarvam-m’s overall delta drops from +36.5% to +26.0%.

D Reproduction Guide

D.1 Required environment

The verification and benchmarking scripts require Python 3.9+ with:

```
tokenizers >= 0.15      # HuggingFace tokenizers library
transformers >= 4.30    # for AutoTokenizer loading
tiktoken                # for o200k_base loading
pandas                  # for CSV output
```

On an M1 Mac with 8-core parallelism, tokenizing the 27M-document Indic corpus takes approximately 80–120 minutes per tokenizer.

D.2 BrahmicTokenizer-131K artifact

Download from HuggingFace: <https://huggingface.co/theschoolofai/BrahmicTokenizer-131K>, or from GitHub: <https://github.com/theschoolofai/BrahmicTokenizer-131K>.

D.3 Smoke test: 4-script verification

Run the smoke test from Listing 1. Expected output: 4 PASS lines, exit 0.

D.4 Reproduce the 14-tokenizer comparison

```
python verify_kronecker_constraints_unified.py
```

Downloads each tokenizer artifact from HuggingFace (3 GB total). Runtime: 5–10 minutes.

D.5 Reproduce the FLORES-200 + IN22-Gen evaluation

```
python evaluate_fertility_flores.py --tokenizers all --languages all
python evaluate_fertility_in22.py --tokenizers all --languages all
```

Runtime: 20 minutes per script.

D.6 Reproduce the 27M-document Indic comparison

```
python tokenize_full_indic_corpus.py --tokenizer brahmic
python tokenize_full_indic_corpus.py --tokenizer tekken
python tokenize_full_indic_corpus.py --tokenizer sarvam1
python tokenize_full_indic_corpus.py --tokenizer sarvam30b
```

Each run: 80–120 minutes on M1 Mac.

D.7 Reproduce code/math compression

```
python evaluate_code_math.py
```

Runtime: 5 minutes for all 14 tokenizers.

D.8 Reviewer-time expectation

- **5 minutes:** 4 verification scripts (Section 4.5) and 14-tokenizer comparison.
- **30 minutes:** FLORES-200 fertility check.
- **2 hours:** HumanEval/MBPP/GSM8K reproduction.
- **8 hours:** 27M-document corpus reproduction.

Full reproduction of all paper claims requires 10 hours of compute on a single M1 Mac. The verification scripts (10 minutes total) are sufficient to confirm the structural claims.